

Kernel Trick for the Cross Section

Detailed Title

We would like to integrate a custom CUDA kernel such that it essentially becomes a native Jax function. That is, the CUDA kernel can be called from Jax and can support Jax features such as JIT compilation, vmap batching, and most importantly autodifferentiation. This tutorial is meant for quant researchers and machine learning researchers looking to efficiently run rigorous experiments.

Introduction

In this tutorial, I'd like to present how to integrate a custom CUDA kernel into Jax such that it essentially becomes a native Jax function. CUDA is a parallel computing platform developed by NVIDIA, and Jax is a Python library for machine learning and high performance computing developed by Google. Some common use cases of CUDA are functions involving lots of matrix multiplication operations because it's very easily parallelizable ie. the element in the resulting matrix is only dependent on the i-th row in one matrix and the j-th column in another matrix. Integrating the two would harness the powers of CUDA and parallel computing into a Python function that can be called from Jax and can support Jax features such as just-in-time compilation, vectorized map batching, and autodifferentiation. This tutorial is primarily aimed at machine learning or quantitative researchers - either in academia or industry - who need to efficiently run enormous experiments. In terms of the level of difficulty for this tutorial, a high level of programming and mathematics experience is required, and this high barrier to entry is primarily due to CUDA, which is not an easy framework to grasp. From experience, finance professors at UMD have been following similar frameworks to this tutorial in their research. Using CUDA and Jax is more desirable than working with Numpy and Pandas, for example, because Numpy is not optimized for GPUs and working with high dimensional tensors in Pandas DataFrames is clunky and cumbersome. In addition, this CUDA/Jax function has been shown to be around 15x faster than its purely native Jax function counterparts, while maintaining a similar speed to the pure CUDA kernel. My sources and citations are listed below as well as their descriptions.

The two main resources I followed to create this custom Jax op are the following:

- https://jax.readthedocs.io/en/latest/Custom_Operation_for_GPUs.html#gpu-ops-code-listing
- <https://github.com/dfm/extending-jax/tree/main>

The GitHub tutorial writes cleaner code and has some nice explanations for some of lower level Jax ideas that the Jax docs omits. Another source of difference is that the GitHub tutorial uses `CMake` and `scikit-build-core` to expose `lib` as a Python library, whereas the Jax docs just uses a shell script, which I thought was more simpler. This project structure more closely resembles the Jax docs,

but the GitHub tutorial is an excellent resource for better understanding what's going on.

Warnings, Cautions, Common Troubleshooting Issues

bash: nvcc command not found

Add CUDA paths:

```
export PATH="/usr/local/cuda-12.3/bin:$PATH"
export LD_LIBRARY_PATH="/usr/local/cuda-12.3/lib64:$LD_LIBRARY_PATH"
```

Python.h not found when running build.sh

Install newest JAX and python3.11.

Add Python path, e.g.: `export CPATH=/usr/include/python3.8:$CPATH`

I was originally troubleshooting this before I fixed the issue above. If after installing CUDA, and this still occurs, then run the following:

```
sudo apt-get update
sudo apt-get install python3-dev
```

and this should work.

No python-config executable

In the setup, we don't have a `python-config` executable but we do have a `python3-config` executable. `whereis python3-config` told me it was at this path: `/home/james/conda/envs/jax/bin/python3-config`. You could go to this `bin` directory and make a copy of the `python3-config` and name it `python-config` or in the `build.sh` script just add a singular 3 after every `${python-executable}`\$. I've tested both, and they both work.

cuda_runtime_api.h: No such file or directory in #include <cuda_runtime_api.h>

Make sure the CUDA path is correct in `build.sh`. E.g., `/usr/local/cuda-12.4` vs. `/usr/local/cuda-12.3`.

Technical Background

The reader should have a strong background in CUDA programming and programming in Jax (or NumPy, which is very similar.)

Requirements (Materials)

1. Install newest JAX and python3.9:

```
“conda create -n jax python==3.9
pip install --upgrade pip pip install --upgrade “jax[cuda12_pip]” -f
https://storage.googleapis.com/jax-releases/jax_cuda_releases.html ““
```

File Structure

C++/CUDA Code

2. Write the CUDA kernel that you would like to integrate into Jax.
3. Have the following file structure, or something along these lines.

```
|– build.sh
|– ckpcalib
|   |– ckpca.so
|   |– ckpca_kernel.cu
|   |– ckpca_kernel.so
|   |– ckpcalib.cpp
|   |– helper_cuda.h
|   |– helper_string.h
|   |– kernel.h
|   |– kernel_functions.h
|   |– kernel_helpers.h
|   |– kernels.h
|   |– pybind11_kernel_helpers.h
|   |– reduce.h
|– ckpcalib.py
|– kernels_cuda.py
|– kernels_jax.py
|– lib
|   |– ckpca_kernel.cu.o
|   |– ckpcalib.cpp.o
|   |– ckpcalib.cpython-39-x86_64-linux-gnu.so
|– pallas.ipynb
|– smap.py
```

|– test__jax_cuda__james.py

Procedure

Let's break this down. Before we start writing any Python code such as defining our custom Jax primitive or abstract evaluation rules, we have to build our `lib` Python library by modifying our existing C++/CUDA code. Specifically, we need to define 2 things:

4. Our Python library has to know which C++/CUDA function to call when we call some function in Jax. There has to be some way to register and link these two together. Using Pybind11, a dictionary keyed by the name of the Python function that points to the address of the CUDA function does just this. See `ckpcalib.cpp` for the implementation.
5. Not only does our library need to know which CUDA function to call, there must exist a way for us to preserve our parameters when they get transferred from Python to CUDA. We do this in the form of a descriptor. The Gaussian Kernel descriptor is defined in the `kernels.h` file. Thankfully, Pybind11 helps us abstract away a lot of even lower level details. `pybind11_kernel_helpers.h` and `kernel_helpers.h` contains a lot boilerplate code for packing and unpacking descriptors which is very helpful. Wrapping them in the `ckpcalib` namespace allows these functions to be accessible anywhere in the `ckpcalib` namespace which is defined across multiple files. In this `kernels.h` file, you can also see the `gaussian_kernel` function which is implemented in the CUDA file and simply launches our CUDA kernel. Consider the parameters of this function:
 - `cudaStream_t stream`: we don't use this at all, but I suspect it's required for the Jax interface
 - `void **buffers`: a double pointer consisting of pointers to the inputs of the function and then the outputs of the function. The implementation in the CUDA file should make this more clear.
 - `const char *opaque`: our kernel descriptor
 - `std::size_t opaque_len`: required to unpack the kernel descriptor (`opaque`)
6. Once these are defined properly, run the `build.sh` script to initially build `lib` library. At this point, however, we have not yet fully implemented all components of the Jax interface such that our custom Jax op supports all of the Jax transformations (`jit`, `vmap`, `grad`).

Python/Jax Code

Everything from this point on (excluding testing) is defined in `ckpcalib.py`. At the top of this file, you will see the use of the `get_kernel_registrations()` function that used a dictionary. This code registers the function name

`gaussian_kernel` with the address of the `gaussian_kernel` CUDA function. Now we implement the Jax interface.

7. Next you see the `gaussian_kernel` function and some variable using the `bind` function. This function is the Primal Evaluation Rule. This variable is called a Jax primitive. There are existing Jax primitives, but we are defining our own. We are implementing the interface **for the Jax primitive** such that that primitive supports all Jax transformations. See the official Jax docs here: https://jax.readthedocs.io/en/latest/notebooks/How_JAX_primitives_work.html.
8. Jax also requires an Abstract Evaluation Rule to support JIT compilation. This function essentially just defines the output shapes of our returned items given the input shape of our parameters.
9. We also need a lowering rule to provide an MLIR “lowering” of our primitive. This provides a mechanism for exposing our custom C++ and/or CUDA interfaces to the JAX XLA backend. XLA compiles and runs Numpy programs on GPUs and TPUs. XLA includes a custom call operation that can be used to encapsulate arbitrary functionality defined using C++ and CUDA. MLIR defines this lowering. Note that our lowering rule is specific for the GPU. In the `custom_call` function note
 - operands are passed into the buffers double pointer from before, and the results are placed into the end of buffers
 - our kernel descriptor is to the CUDA function as `backend_config=opaque`
 - layouts are defined for parameters and return variables
 - we also have to define our result types given input types
10. Benchmark performance between the CUDA code, CUDA kernel integrated into Jax, and the native Jax code to ensure that your code is actually faster and more efficient.

Five Sources Directly Related To My Topic

1. Dfm. (n.d.). GitHub - dfm/extending-jax: Extending JAX with custom C++ and CUDA code. GitHub. <https://github.com/dfm/extending-jax/tree/main>

The GitHub tutorial writes cleaner code and has some nice explanations for some of the lower level Jax ideas that the Jax docs omit, and will provide an excellent example to someone following my tutorial. This GitHub tutorial uses CMake and scikit-build-core to expose lib as a Python library, whereas the Jax docs just uses a shell script, which I thought was simpler. This project structure more closely resembles the Jax docs, but the GitHub tutorial is an excellent resource for better understanding what’s going on. I found this tutorial when just searching up “Extending Jax with CUDA,” and this link was the first one to appear, even before the official Jax docs, which was quite interesting.

2. Google Colab. (n.d.). <https://colab.research.google.com/github/dfm/extending-jax/blob/main/demo.ipynb>

This resource is an extension of Citation 1, and it was linked at the bottom. This Google Colab script provides all of the code and file structure needed to integrate a CUDA kernel into native Jax – which will definitely be useful to the follower of the tutorial – although the function that they integrate is far different from the one that I want to use. They do implement the same concept, however, and there aren't that many examples on the internet of people doing this, so I think it would be beneficial for the follower of my tutorial to see multiple examples and codes.

3. Custom operations for gpus with C++ and Cuda. Custom operations for GPUs with C++ and CUDA - JAX documentation. (n.d.). https://jax.readthedocs.io/en/latest/Custom_Operation_for_GPUs.html#gpu-ops-code-listing

This source is the official documentation for creating Custom Operation for GPUs on the Jax documentation page. To expose lib as a Python library, these docs use a straight forward shell script, and I will closely follow the file structure of this tutorial, rather than the GitHub repository. I found this documentation and tutorial through Citation 1, which is the above GitHub repository.

4. IRIS-HEP. (2021, February 2). IRIS-HEP Topical Meeting (1 Feb 2021) - Extending JAX with CUDA [Video]. YouTube. <https://www.youtube.com/watch?v=NJHEPV4Etmg>

I found this video on YouTube, and it is a recorded lecture of someone presenting a similar concept at a virtual conference. The presenter does an excellent job of not only explaining how to implement the various components of the Jax interface, but they also detail design choices of lower level Jax, and I wouldn't be surprised if they worked at Nvidia or Google. I will explain concepts like abstract evaluation and transformation rules in a similar manner to this presenter.

5. 650 AI Lab. (2022, July 6). JAX installation with Nvidia CUDA and cudNN support (Fixing most common installation error) [Video]. YouTube. <https://www.youtube.com/watch?v=auksaSl8jIM>

I also found this video on YouTube, and I think that an overlooked component of programming and coding is the environment setup. If one were to follow my tutorial, they would most certainly need to configure a virtual environment where not only CUDA and Jax are installed correctly, but the two libraries also must be compatible, and this is actually not an easy task. Not only do there need to be specific versions for both, there are also an assortment of other libraries Jax and CUDA depend on, most notably JaxLib.

Three Context Sources

6. Jax internals: Primitives. JAX Internals: primitives - JAX documentation. (n.d.). <https://jax.readthedocs.io/en/latest/jax-primitives.html>

This source provides some more background Jax primitives and operations as well as the interface that one must implement in order to create their own custom operation. This is recommended reading for our tutorial, but it is not required. Next you see the `gaussian_kernel` function and some variables using the `bind` function. This function is the Primal Evaluation Rule. This variable is called a Jax primitive. There are existing Jax primitives, but we are defining our own. We are implementing the interface for the Jax primitive such that that primitive supports all Jax transformations. I found this website linked in Citation 2, as it provided a bit of background reading and insight into the Jax interface.

7. Just-in-time compilation — JAX documentation. (n.d.). <https://jax.readthedocs.io/en/latest/jit-compilation.html>

This source provides more information about the idea of “just-in-time compilation.” Python is interpreted, but can be sped up if it is compiled before running. Many packages do this, but Jax does it the best because they can also optimize the compiled code for GPU execution, whereas most libraries only optimize for CPU execution. I found this website linked in Citations 1 and 6, as this is one of the main reasons why people choose to use Jax over traditional Python libraries such as NumPy and Numba.

8. Fireship. (2024, March 7). Nvidia CUDA in 100 seconds [Video]. YouTube. <https://www.youtube.com/watch?v=pPStdjuYzSI>

I found this short video on YouTube. Fireship is an extremely popular technology and programming focused YouTuber who frequently posts short informative videos on recent technologies - I actually saw this video before coming up with the idea to make this tutorial, as I was extremely curious on how CUDA worked and its applications. My tutorial assumes that the reader is also familiar with CUDA, but if they are not, then they can watch this video to become introduced to its basic concepts.